CO1 - AT2

Name : Sruthi Rithika. P
Reg NO : 192521407
Course code : CSA1406
Course : compiler design
Date : 2.8.26

1) Error Analysis Task

a) Error Identification

| Line | Error | Type |
|---|---|---|
| 5 | i<=n should be i<n | Logical |
| 11 | Missing ; | syntax |
| 13 | sumArray(arr) | semantic |
| 15 | Missing ; after printf() | syntax |
| 18 | Division by zero (a/b) | Runtime |
| 20 | uninitialized pointer (*p = 20) | Runtime |

b) classification

* Lexical : No lexical errors
* syntax : Missing semicolons
* semantic : wrong number of function arguments
* Logical : wrong loop condition (<=)
* Runtime : Division by zero, invalid pointer access.

c) compilation Behavior

* compile-time errors : Missing semicolons, function argument mismatch.

* Run-time errors : Division by zero, invalid pointer, array out-of-bounds.

d) code correction

* change i<n to i<n.
* Add missing semicolons.
* call sumArray(arr, 5);

```
int value = 20;
int *p = &value;
```

e) Impact Analysis

* Pointer error : causes segmentation fault.
* Division by zero : Program crashes or gives undefined behavior.
* Wrong loop condition : Reads memory outside the array.

f) compiles Improvements.

* Better error message with suggestions.
* Static analysis to detect division by zero and pointer errors early.

2) Lexical Errors

a) Error Identification

Invalid Token

1 num

rate@

'abc'

$

num#1

%. count

"

09

0x212

12.3.4

"Hello

invalid-id

5..6

10@5

'xy'

3value

b) Classification

* Identifier errors : 1num, 3value, rate@, num#1
* constant errors : 09, 0x712, 12.3.4, 5..6
* string/character errors : 'abc', 'xy', missing quotes
* operator errors : $, @

c) Token Correction

* 1num → num1
* rate@ → rate
* 'abc' → 'a'
* 09 → 9
* 0x712 → 0x12
* 3value → value3

d) Compiler Behaviour

* scanner reads source code and converts it into tokens.

* It reports lexical errors and continues scanning using error recovery.

e) Advanced Thinking

DFA Rule: Identifiers must begin with letter or underscore, not a digit.

Error messages should include the line number and a correction suggestion.

3) Multi-Phase Error Analysis

a) Error Identification

| Error | Type |
|---|---|
| Missing semicolon after return. | syntax |
| # value | Lexical |
| Missing semicolon after 15.5 | syntax |
| divide (10) | semantic |
| if (result = 5) | Intermediate code / Logical |
| printf (missing semicolon) | syntax |
| Division by zero | Runtime |
| arr [4] | Runtime |
| *ptr = 25 | Runtime |
| +0 | optimization issue |
| infinite loop | Runtime |

b) classification

* lexical : # value
* syntax : missing semicolons
* semantic : wrong function arguments
* Intermediate code : Assignment (=) instead of comparison (==)
* optimization : Redudant +0

c) compilation vs Execution

* compile - time : Lexical, syntax, semantic errors.
* Run - time : Division by zero, invalid pointer, array index error, infinite loop.
* lexical and syntax errors stop successful compilation.

d) code correction

* Add missing semicolons.
* Chang # value to value.
* call divide (10,2);
* change = to ==
* use arr[3].
* Initialize pointer
* Remove +0

e) Deep Analysis.

* Pointer error : May crash the program
* Array overflow : Accesses invalid memory
* Assigment instead of comparison : Produces incorrect program logic.

f) compiles Improvements

* use static analysis to detect runtime errors early.
* Improve error messages with suggestions and exact locations.
* Rerame redundant expressions (eg :+0) during optimization.